

Recovery Desk

System Design Document

Author Sahaj Gaur

Version v0.1 — design lock, pre-build

Submission 5 September 2026

Status Direction locked. Scope open until Day 2.

Razorpay shipped agents that recover revenue. This is the desk that decides which revenue is worth recovering — and proves it worked.

CONTENTS

1. Summary and positioning
2. The problem model
3. Non-goals and scope discipline
4. System architecture
5. The decision core
6. Where AI belongs, and where it does not
7. Guardrails and bounded action
8. Data model
9. The evaluation harness
10. Failure taxonomy
11. Interfaces and repository layout
12. Assumptions to verify on Day 1
13. Panel defence notes

1. Summary and positioning

Recovery Desk is an agent that finds revenue slipping away from a merchant, decides which of it is worth chasing under a finite recovery budget, executes bounded recovery actions, and then proves what it recovered against honest baselines.

The distinguishing claim is not that it recovers revenue. It is that it knows which revenue not to chase, and can show the arithmetic.

Why this shape

At FTX'26 Razorpay launched Agent Studio, built on Anthropic's Claude Agent SDK, and shipped four production agents: Abandoned Cart Conversion, Subscription Recovery, Dispute Responder, and Cashflow Forecaster. The buildathon tracks map onto that product line.

Two consequences follow, and they define this design:

- Rebuilding any of those four agents means submitting a weaker copy of a product the judges shipped. This design deliberately avoids all four.
- Those agents are point solutions. Nothing above them decides where a merchant's finite recovery effort should go, and nothing measures whether any of it worked. That is the gap this fills.

Recovery Desk is therefore complementary to Razorpay's direction rather than competitive with it, and it is built on the same foundation — the Claude Agent SDK — so the architecture is legible to the people evaluating it.

The one insight

Roughly a third of payment failures are structurally unrecoverable. Any agent that treats all failures as equally worth chasing burns money on outreach that could never have worked, and burns customer patience alongside it. Recovery is an allocation problem under constraint, not a messaging problem.

2. The problem model

At-risk revenue arrives as a heterogeneous pool: failed payment attempts, subscription charges that will fail, carts abandoned before payment, invoices past due. Each item carries an amount, a cause, and a recovery cost profile.

Recoverability is not uniform. Published merchant-side failure distributions look approximately like this:

Failure class	Share	Recoverability	Correct action
Bank / PSP timeout (technical decline)	35–45%	High	Fast retry, then alternate rail
Wrong PIN / attempts exceeded	20–30%	Low by retry	Offer another method; do not chase
Insufficient balance	15–25%	Time-dependent	Retry on the salary cycle, not on backoff
Network / connectivity	10–15%	Moderate	Short-delay retry

Failure class	Share	Recoverability	Correct action
Account blocked or frozen	5–10%	None	Suppress — spending here is pure waste

Blended merchant success rates sit around 92–96%. NPCI targets technical decline below 1% and business decline below 5%. Industry smart-retry logic recovers roughly 20–30% of timeout failures — that number is the bar the agent must beat, and it is the reason a naive retry baseline must appear in the evaluation.

The salary-cycle insight

Insufficient-balance failures are 15–25% of the pool and are usually treated as a fixed exponential-backoff problem. They are not. Balance is a calendar phenomenon: it recovers on payday. A retry scheduled against salary-credit patterns and known issuer downtime windows should materially outperform a retry scheduled against an exponential curve.

This is the single sharpest rule in the system, it is specific to the Indian market, and it is directly measurable against the naive baseline.

Note on sourcing. The distribution above currently rests on a single industry source. A second independent source must be found before any of these figures appear on camera or in the README.

3. Non-goals and scope discipline

Scope discipline is a design feature here, not an admission. The build window is ten days, the panel interview tests whether every boundary can be defended, and the most common failure mode of AI-assisted projects is four half-verified loops instead of one airtight one.

Explicitly not building

Excluded	Reason
Abandoned cart conversion bot	Razorpay ships this (Abandoned Cart Conversion Agent)
Subscription dunning bot	Razorpay ships this (Subscription Recovery Agent)
Dispute / chargeback responder	Razorpay ships this (Dispute Responder Agent)
Cash-flow forecasting	Razorpay ships this (Cashflow Forecaster Agent); also Track 4
Fraud detection and scoring	Track 2, and no credible labelled data exists for it
A merchant-facing notification channel for e-mandates	The pre-debit notification is the issuer's obligation, not the merchant's. A merchant-side agent cannot own that channel.

Deliberate limits inside scope

- One leak class is implemented deeply — failed payment attempts. A second detector (subscription pre-failure) is added only to demonstrate that the architecture generalises, and only after the first is complete and scored.

- The agent proposes and executes recovery actions. It never moves money autonomously.
- No user interface beyond a results view. The deliverable is the decision quality and the evidence, not the chrome.

4. System architecture

Five stages, one direction of flow. Each stage has a single responsibility and a typed boundary, so any stage can be swapped or ablated in the evaluation harness without touching the others.

#	Stage	Responsibility	Output
1	Ingest	Pull at-risk items from the transaction stream and subscription schedule; normalise to a common shape	AtRiskItem[]
2	Diagnose	Classify the failure cause from raw gateway responses; attach a recoverability prior	Diagnosis
3	Decide	Enumerate candidate actions, price them, rank by expected value, allocate under budget and policy caps	Decision
4	Act	Dispatch the chosen action idempotently through the policy gate; record the outcome	ActionAttempt
5	Prove	Score the batch against baselines; emit metrics, regressions and the exception list	BatchReport

Stage 5 is not an afterthought bolted on at the end. It is the reason the product exists, and it is built on Day 6 rather than Day 10.

Flow

```

transaction stream + subscription schedule
    |
    [ 1 INGEST ]      -> AtRiskItem[]
    |
    [ 2 DIAGNOSE ]    -> failure class + recovery prior
    |
    [ 3 DECIDE ]      -> EV rank, allocate under budget
    |                  (incl. the option: do nothing)
    < policy gate >    caps - dedupe - idempotency
    |
    [ 4 ACT ]         -> retry / reroute / notify / escalate
    |
    [ 5 PROVE ]       -> scored vs B0 / B1 / B2

```

5. The decision core

For each at-risk item and each candidate action, the desk computes an expected value:

```
EV(item, action) =  
    P(recover | item, action) x amount x margin  
    - cost(action)  
    - lambda x contact_fatigue(customer, action)
```

Each term is deliberately simple and inspectable:

Term	Where it comes from
$P(\text{recover} \mid \text{item}, \text{action})$	Base rate by failure class (a published table), adjusted for context — amount band, issuer, hour of day, position in the salary cycle, prior attempts on this customer
$\text{amount} \times \text{margin}$	The item's value to the merchant, not its gross value. Recovering a ₹1,000 order at 20% margin is worth ₹200 of effort, not ₹1,000.
$\text{cost}(\text{action})$	Gateway fee for a retry, message cost for a nudge, per-minute cost for a voice call. Real numbers, configurable.
contact_fatigue	A penalty that grows with each touch on the same customer, tuned by lambda. This is how the system prices the customer relationship rather than ignoring it.

Allocation

Given a batch and a recovery budget B , the desk selects the action set that maximises total EV subject to three constraints: total spend must not exceed B , no customer may exceed the contact cap in a rolling window, and no transaction may exceed its retry cap.

Selection is greedy on EV-per-rupee-spent. This is a deliberate choice over a fancier optimiser: greedy is $O(n \log n)$, explainable line by line to a panel, and produces a decision trace a human can audit. A more sophisticated solver would buy a small amount of optimality at the cost of the property that matters most here — being able to say exactly why item 447 was chosen and item 448 was not.

"Do nothing" is a first-class action with cost zero and EV zero. Any item whose best action scores below zero is suppressed, and suppression is recorded with its reason. The count of deliberately suppressed items is a headline metric, not a hidden detail.

6. Where AI belongs, and where it does not

Razorpay's stated evaluation criteria penalise using AI where deterministic logic would serve. This table is the answer to that criterion, and it should be reproduced verbatim in the README.

Concern	Owner	Reasoning
Retry eligibility and backoff windows	Rules	Issuer policy is fixed and knowable; behaviour must be predictable and auditable
Spend caps, contact caps, deduplication	Rules	Safety must never depend on a sampler

Concern	Owner	Reasoning
Idempotency	Rules	A correctness primitive, not a judgement call
Failure classification from raw gateway text	Model	Unbounded text with a long tail of formats; regexes rot
Recovery probability	Both	Published base rates as the prior; model adjusts for messy context
Action selection	Rules	Deterministic arithmetic given the inputs; must be explainable
Customer message drafting	Model	Tone, language and localisation across Indian languages
Edge-case escalation	Model proposes, rules gate	Judgement where rules are brittle — but never unbounded
Audit narrative for humans	Model	Explanation is a language task
Money movement	Rules only	Non-negotiable. No model output initiates a debit or refund.

An ablation is required: the harness must be able to run the pipeline with the model stages replaced by their deterministic fallbacks, so the contribution of the model can be stated as a number rather than asserted.

7. Guardrails and bounded action

The track brief requires every money action to be explainable, bounded, gated and audited. That is treated here as a scored requirement, not a footnote.

Hard bounds

- Dry-run is the default mode. Live dispatch requires an explicit flag and a confirmed budget.
- Maximum retries per transaction, enforced before dispatch.
- Maximum contacts per customer per rolling window, across all channels.
- Total spend ceiling per batch, checked before each dispatch rather than after.
- An idempotency key on every action, so a replayed batch cannot double-charge or double-message.
- A closed enumeration of permitted actions. Any model output that does not resolve to a member of that set is rejected at the gate, logged, and counted.
- A kill switch that halts dispatch mid-batch and leaves the ledger consistent.

Audit record

Every decision writes a row containing: the item, the diagnosis and its evidence, every candidate action with its priced EV, the chosen action or the suppression reason, which component produced each input (rule or model, and which model version), the policy checks applied, and the outcome. The audit log is a first-class output, queryable, and shown in the video.

Explicit refusals

The system is designed not to do the following, and the README says so:

- It will not split transaction amounts to fall below an authentication threshold. That is regulatory evasion.
- It will not suppress, delay or obscure a customer's opt-out. That is a dark pattern.
- It will not misrepresent urgency, consequences or account status in a recovery message.
- It will not retry against an account flagged as blocked or disputed.

Stating these unprompted is itself a signal. It maps directly onto the value Razorpay describes as agility with integrity.

8. Data model

Entity	Fields
AtRiskItem	id · type (payment_failure subscription_due invoice_overdue) · amount · currency · merchant_id · customer_id · occurred_at · source · raw_gateway_context
Diagnosis	item_id · failure_class · confidence · evidence · recovery_prior · classifier_provenance
ActionCandidate	item_id · action_type · estimated_cost · estimated_p_recover · earliest_executable_at
Decision	item_id · chosen_action suppressed · ev · full_ev_table · rationale · provenance · policy_checks_passed
ActionAttempt	decision_id · idempotency_key · dispatched_at · outcome · amount_recovered · cost_incurred
BatchRun	id · fixture_id · policy_version · budget · totals · metrics · started_at · finished_at

Every entity is immutable once written. Corrections are new rows, not edits. This makes the audit trail trustworthy and makes replaying a batch against a new policy version trivial.

9. The evaluation harness

This is the differentiator. In 2026 any student can wire a model into a workflow; almost none can demonstrate whether the result is working. The harness is what converts a demo into evidence.

Fixtures

A seeded generator produces a fixed set of at least 1,000 at-risk items with known ground truth — for each item, what would have happened under each candidate action.

The circularity trap. The simulator's generative process must be independent of the agent's feature set. If the agent scores well because it reads the same variables the simulator used to decide outcomes, the

number is meaningless. The separation must be documented explicitly in the README, and the panel will ask about it.

Baselines

ID	Baseline	Why it is in the harness
B0	Do nothing	The floor. Establishes how much revenue is genuinely at stake.
B1	Blanket retry ×3, fixed backoff	What most merchants actually do, and what most competing submissions will implement
B2	Rules-only heuristic	The best static policy without any model. This is the honest bar.
B3	Recovery Desk	The agent under test

B2 is the one that matters. If the rules-only policy captures most of the agent’s result, that must be reported plainly, along with an account of exactly where the model earned its place. Reporting that honestly is the strongest available signal of engineering maturity, and concealing it is the fastest way to lose a panel that will find it anyway.

Metrics

Metric	Definition
Gross recovered	Total value of items successfully recovered
Net recovered	Gross recovered minus all action costs incurred
Recovery rate	Share of genuinely recoverable value that was captured
Wasted spend	Cost expended on items that never recover — the number nobody else will report
Chase precision	Share of items chosen for action that were actually recoverable
Contacts per rupee recovered	Proxy for customer annoyance; lower is better
Cost per rupee recovered	The efficiency headline
p95 decision latency	Throughput evidence at batch scale
Policy violations	Must be zero. Any non-zero value is a build failure, not a metric.

Regression tracking

Every scored run appends a row to a versioned results table, including the runs that got worse. That table goes in the README. A monotonic improvement curve reads as fabricated; a real one with visible regressions and an explanation of each reads as engineering.

10. Failure taxonomy

The submission form has a required field — Build Challenges and Technical Obstacles — asking what broke and how it was solved. It is one of only three substantive text fields, which means roughly a third of the written submission is about failure. That is a deliberate probe, and it maps onto the values Razorpay interviews for: transparency, and agility with integrity.

Keep a running log from Day 1, structured per entry:

Field	Content
Symptom	What was observed, with the actual error or the wrong number
Wrong hypothesis	What it was assumed to be first, and why that was reasonable
Root cause	What it actually was
Fix	The change, and what it cost
Guard	The test or invariant added so it cannot recur silently

The most valuable entries are the ones where an AI-generated implementation looked correct and was not — a silent double-dispatch, an idempotency key that was not actually unique, a metric computed over the wrong denominator. Those are the entries that demonstrate the diagnosis skill the criterion is testing.

The system must also carry an honest list of unresolved exceptions: the failure classes it still handles badly, with a stated reason for not solving them inside the window. Claiming near-total success invites a panel to find the hole; naming the hole first removes the weapon.

11. Interfaces and repository layout

The submission form has no architecture-document field. The README therefore is the architecture document, and it carries the entire written weight of the technical case alongside the video.

One-command reproduction

A judge triaging hundreds of repositories will spend between sixty and ninety seconds deciding whether to watch the video. The repository must clone and run on a stranger's machine with a single command, print the headline result table, and exit cleanly.

```
make demo    # seeded batch of 1,000 items, all baselines, results table
make eval    # full harness, regression table, exception list
make test    # unit and policy-gate tests
```

Most student repositories do not run for the person cloning them. Satisfying this single property likely beats the majority of the field on the criterion Razorpay weights most heavily — that the thing actually works.

Layout

```

recovery-desk/
  README.md      architecture, results, honest limits
  Makefile       demo / eval / test
  src/
    ingest/      stream and schedule adapters
    diagnose/    classifier + deterministic fallback
    decide/      EV model, allocator, policy gate
    act/         dispatchers, idempotency, audit log
    prove/       harness, baselines, metrics, reports
  fixtures/      seeded generators + ground truth
  results/       versioned run tables (committed)
  docs/
    decisions.md why each boundary is where it is
    failures.md  the build failure log

```

Repository hygiene

- Incremental commits with real messages. Twelve large dumps of generated code is a visible tell.
- One consistent point of view across modules — no orphaned helpers, no dead abstractions, no docstrings that contradict behaviour.
- The submission is final and cannot be edited. The repository must be quiet and complete before the form is submitted; no force-pushes or renames afterwards.

12. Assumptions to verify on Day 1

Each of these is cheap to resolve now and expensive to discover late.

Assumption	How to kill it early
Razorpay test mode can produce or simulate the failure cases needed	Spike the test-mode API on Day 1. If it cannot, the synthetic generator carries more weight and must be more defensible.
The failure-mix distribution is accurate	Find a second independent source before any figure goes on camera
Real merchants will talk within two days	Start outreach on Day 1. Fallback: Indian founder communities and public D2C operator threads.
The Claude Agent SDK fits the batch-processing shape	Build the thinnest end-to-end skeleton on Day 2, before any depth

Evidence layer

Customer First is the first value Razorpay names in its hiring philosophy, and problem validity is the first criterion in the buildathon rubric. Nearly every submission will assert the problem is real; almost none will evidence it.

Five conversations with real merchants — a D2C store, a SaaS founder, a subscription business — asking one question: where does money go missing every month. One recorded sentence from a real merchant opening the video is worth more than any synthetic framing, and it satisfies the top-weighted criterion with evidence instead of assertion.

13. Panel defence notes

The panel interview is where the offer is made. Every boundary in this document must be defensible from memory on a blank page. If any part cannot be, cut it until it can.

Likely question	The answer this design supports
Why is there a model here at all?	Point at the ownership table in section 6, then quote the ablation number showing what the model actually contributed
How do you know your synthetic data is not circular?	The simulator's causal variables are disjoint from the agent's features, documented in the README
What if the model proposes a nonsense action?	The action space is a closed enumeration; anything outside it is rejected at the policy gate, logged and counted
Where did the model make things worse?	The regression table, including the runs that went backwards
How does this behave at ten million transactions a day?	The decision core is greedy and $O(n \log n)$; model calls occur only at diagnosis and are batched and cached by failure signature
What is the cost of a false positive to the merchant?	Wasted spend and contacts-per-rupee are reported as headline metrics, not buried
Which part would you rewrite?	Have a real, specific answer prepared — a vague one reads as not having understood the code

Nothing in this document is load-bearing on a claim that has not been checked. Where a figure rests on a single source, that is stated. Where an assumption is unverified, it appears in section 12 with a plan to kill it on Day 1.